

🎤 AI 實戰分享 | 25 分鐘講稿

目標時長：25 分鐘以內（實測配置 ≈ 24 分鐘，留 1 分鐘緩衝）**標示說明**：▶ = 跳過 / 一句帶過 ▶ = 播放影片頁 影片頁讓影片自己講，你只需要起頭 + 收尾各一句，別在播放時蓋話。

🕒 時間配置總表（總預算 1500 秒）

頁	內容	處理	秒數
1	封面 言出法隨	講	40s
2	自我介紹	講（精簡）	50s
3	打字 vs 說話 45/220	講	50s
4	Typeless 三步驟	講	55s
5	Typeless 演示影片	▶ 播片	60s
6	別當聊天室·當執行助理	講	70s
7	想→寫→做 三段工作流	講（精簡）	55s
8	Skill/Hook/Cron	講（快帶）	50s
9	Codex 拆解案例·開場	▶▶ 快速過	25s
10	Model ≠ Agent	講（核心）	65s
11	4 種介面	▶▶ 一句帶過	15s
12	6 種核心模組	▶▶ 一句帶過	15s
13	五層工作流	講（精簡）	45s
14	方法可遷移	▶▶ 一句帶過	15s
15	Case01 痛點 part number	講	50s
16	Case01 解法終端機	講（精簡）	40s
17	Case01 iframe 雙版 demo	▶▶ 快速展示	35s
18	Case01 成果 KPI	▶▶ 一句帶過	15s
19	Case02 旅遊助手起心動念	講	50s
20	Case02 Gemini→規格書→Codex	講（核心）	70s
21	Case02 成品 + QR	講（精簡）	40s
22	Case02 iframe 雙版	▶▶ 跳過	10s
23	Case03 修電腦場景	講	45s

頁	內容	處理	秒數
24	Case03 修電腦影片	▶ 播片	60s
25	Case03 4 動作精煉	▶▶ 一句帶過	15s
26	Ch6 章扉 不只工具是助理	講	35s
27	OpenClaw 機場追星	講 (故事)	70s
28	Hermes Lemonade 演示影片	▶ 播片	60s
29	整合關係圖	講 (精簡)	40s
30	心法 epilogue	講	40s
31	本週試試看 3 件事	講	45s
32	QR + 致謝 Q&A	講	30s

合計 ≈ 1438 秒 ≈ 24 分鐘

【P1・封面】言出法隨·幻化成真 ⌚ 40s

大家好，我是 Andy。今天分享的標題叫「言出法隨、幻化成真」——聽起來很玄，但講的是一件很實在的事：**現在你用嘴巴講一句話，電腦真的會幫你把事情做完。**

過去一個月，我靠這四個工具——Typeless 語音輸入、Claude Code、Codex、還有 Hermes 和 OpenClaw 這兩個 Agent——做了好幾個以前我覺得「我不可能做得到」的東西。今天就把這套方法，完整分享給各位。

【P2・自我介紹】⌚ 50s

先快速自我介紹。我是傅俊諺，在 RBU 部門做 PCB 客服工程師，CSE。

我不是工程師背景，物理系畢業，做客服十一年。但我有個特點：**我是一個有軟體思維的硬體工程師**——平常除了顧 LDI 機台，也會自己寫小工具幫團隊備份、還原資料，順便當部門的電腦疑難雜症窗口。

下班後我是個極客：自己架 VPS、玩路由器、架 VPN 和網站，最近最熱衷的就是研究 AI 最新技術。今天分享的就是這一個月的實戰成果。

【P3・打字 vs 說話】⌚ 50s

我想先問一個問題：**為什麼我們到現在，還在用十根手指，去追自己的思想？**

看這個數字——打字平均每分鐘 45 個字，但說話可以到 220 個字，差不多是五倍。而 AI 一秒鐘就能讀完一千個字。

所以問題來了：當對面這個 AI 讀得這麼快、你講得又比打得快五倍——**那你為什麼還在打字？** 這就是我這套方法的第一塊基石：把輸入，從手指換成嘴巴。

【P4 · Typeless 介紹】 ⌚ 55s

我用的工具叫 **Typeless**。用法簡單到只有三步：

1. **按住快捷鍵**——它是全域的，任何輸入框都能用，不挑 App。
2. **對著筆電講**——你可以結巴、可以重來、可以中英文夾雜，就像跟人講話一樣。
3. **放開，文字自動貼上**——Whisper 等級的中文辨識，Slack、Cursor、Codex、ChatGPT 通通能用。

而且它桌面跟手機介面一致，macOS、Windows、iOS 都有。學一次，到處用。

【P5 · Typeless 演示影片】 ⌚ 60s 播片

【起頭】講再多不如直接看。這段影片是真實畫面——我按住快捷鍵、講話、放開，文字就自己出現了。各位注意看辨識速度跟準確度。

(播放影片)

【收尾】對，就這麼簡單。從今天起，你的鍵盤只是備胎。

【P6 · 別當聊天室·當執行助理】 ⌚ 70s

接下來是觀念上最重要的一頁。**大部分人用 AI，是把它當聊天室——一問一答。**

你看左邊：「寫個 app」→ AI 問「什麼 app？」→「查料件的」→「什麼語言？什麼資料庫？」→最後你「算了」。為什麼會這樣？因為你上下文沒包好，被它反覆反問，而且講完還要自己複製貼上去跑。

真正進階的用法在右邊：**先寫一份規格書，再交給 Agent 工具去執行。**

第一步，在聊天視窗——用 Gemini、ChatGPT、Claude——把需求討論清楚，產出一份 Markdown 規格書。第二步，把這份規格書貼給 Claude Code、Cursor、Codex，**它自己改檔、自己跑指令、自己生網站、自己部署。**

一次跑完，直接給你能用的成品。這就是「聊天室」跟「執行助理」的差別。

【P7 · 想→寫→做】 ⌚ 55s (精簡)

把剛剛那個概念整理成一條工作流，就三個字：**想、寫、做。**

- **想**——在聊天視窗跟 AI 討論，逼它反問你、補你沒想到的細節，順便省 token，因為聊天視窗便宜。
- **寫**——產出一份 Markdown 規格書：角色、目標、限制、範例、驗收條件，變成 AI 的「需求單」。
- **做**——把規格書貼給 Agent，它就去改檔、跑指令、部署上線。

一句話：**先想清楚，再交給 Agent 做。**

【P8 · Skill / Hook / Cron】 ⌚ 50s (快帶)

Agent 還不只是「下指令」這麼簡單，它能自己跑、自己學、自己排班，靠的是這三樣：

- **Skill** 是專業領域包——給它一個包，它就學會那個領域的規矩跟 API。**今天這份簡報，就是用一個叫 html-ppt 的 skill 做的**，一行指令就裝好版型跟動畫。
- **Hook** 是條件觸發——「每當 X 發生，就幫我做 Y」，比如每次存檔自動跑檢查。
- **Cron** 是排程——「每天九點幫我做某件事」，它真的會準時做。

這三個疊起來，Agent 就從「被動的工具」變成「主動的同事」。

【P9 · Codex 拆解案例·開場】 ⌚ 25s ▶ 快速過

接下來這幾頁，我用 Codex 當一個「拆解案例」。為什麼選它？因為**只要把 Codex 拆透了，這套理解方式套到 Claude Code、Hermes、OpenClaw 全部一樣快**。我們快速看幾個關鍵觀念就好。

【P10 · Model ≠ Agent】 ⌚ 65s (核心)

這頁是整場最重要的觀念：**Model 是 Model，Agent 是 Agent。**

中間這顆是「腦」——也就是 Model，GPT、Claude、Gemini 這些都是。Model 只會做一件事：**想**。它會推理、會答題、會產生文字——但它不會動手，只活在對話框裡。換一個 Model，等於換一顆腦。

而 **Agent = Model + 動手能力**。它會讀檔、改檔、跑指令、調工具，有權限邊界、有交付產物。Codex、Claude Code、OpenClaw 都是 Agent。換工具只是換包裝，裡面的模組是一樣的。

記住這一句就好：**Model 會想，Agent 會做。**

【P11 · 4 種介面】 ⌚ 15s ▶▶ 一句帶過

同一個 Codex 有四種介面——手機、桌面、命令列、IDE 外掛——選哪個不是看順手，是看任務形態。這頁細節我跳過，有興趣的線上版可以慢慢看。

【P12 · 6 種核心模組】 ⌚ 15s ▶▶ 一句帶過

這頁是 Agent 圍繞 Model 的六種核心模組：Agent、Skill、Markdown、[AGENTS.md](#)、資料庫、MCP。重點只有一句：**名詞會一直變，但位置就這幾個。看懂位置，就不怕新工具。** 細節一樣留給線上版。

【P13 · 五層工作流】 ⌚ 45s (精簡)

把剛剛那些模組串起來，任何 Agent 工具都跑同一條五層鏈路：

入口——從哪裡發任務；**執行環境**——在哪裡幹活；**上下文**——憑什麼做對；**權限邊界**——能動什麼不能動什麼；**交付**——最後給你什麼成品。

所有 Agent 工具，不管名字多炫，骨架都是這五步。

【P14 · 方法可遷移】 ⌚ 15s ▶▶ 一句帶過

這頁的結論一句話：**Codex 是樣本，方法可以搬走。** 在 Codex 上學會的，直接套到 Claude Code、OpenClaw、Hermes 都一樣。好，理論講完，接下來看真實案例。

【P15 · Case01 · Part Number 痛點】 ⌚ 50s

第一個案例，從我每天的工作講起。我每天要查料號 part number，舊流程是：開 Excel、找到對的工作表、Ctrl+F 搜尋、複製、切視窗、貼上.....重複再重複。

繁瑣、容易出錯。而且老實說，**這件事我以前覺得「沒辦法」**——因為我不是工程師，不會 React、不會後端、不會部署。我以為這種工具，輪不到我自己做。

【P16 · Case01 · 解法】 ⌚ 40s (精簡)

結果呢？**一段需求就搞定。**

我用 Typeless 講了大概 80 個字：「幫我做一個查 part number 的小工具，後端 FastAPI、前端靜態頁，輸入料號顯示資訊就好。」

Codex 就自己把 FastAPI 後端、前端、搜尋邏輯一條龍寫完，還加了密碼保護給同事用。我從頭到尾，只是講了一段話。

【P17 · Case01 · iframe 雙版 demo】 ⌚ 35s ▶ 快速展示

這是實際成品，桌面跟手機同一份資料，都要密碼進站、RWD 自動適應。

(若現場可互動，點一下輸入框查一筆；否則指一下畫面即可)

重點是：這是一個非工程師，用講的做出來、而且每天在用的內部工具。

【P18 · Case01 · 成果 KPI】 ⌚ 15s ▶ 一句帶過

成果一句話：查詢從多步驟變成一個欄位，不用開 Excel、不用切視窗。vibe coding 把「我以為做不到的工具」，變成「用說的就能擁有的工具」。

【P19 · Case02 · 旅遊助手】 ⌚ 50s

第二個案例更生活化。五月我要去成都八天，想要一個客製化的旅遊 App——但市面上沒有完全合我意的。既然沒有，那就自己做一個。

我要的功能很具體：航班自動倒數、地點一點直接跳 Google Maps、行程隨時可編輯、預算自動分類彙總、還有必吃清單。對了，去成都，主題當然要熊貓。

【P20 · Case02 · Gemini→規格書→Codex】 ⌚ 70s (核心)

這頁是整套方法最完整的示範，兩個 AI 接力。

- **第一步**，我先跟 Gemini 聊。我說我要去成都八天、四個城市，想做個手機旅遊 App，請它幫我寫一份詳細到 Codex 能直接照做的規格書。Gemini 反問我五個關鍵問題——行程要不要可編輯、幾個城市、預算分幾類、資料存哪、UI 風格——我一一回答。
- **第二步**，Gemini 產出一份兩千五百字的 Markdown 規格書，詳細到工程師看著就能做。
- **第三步**，我把整份規格書貼給 Codex，它就照著一步步生出單檔 HTML、套熊貓主題、加 localStorage。

為什麼要分兩個 AI？因為規格書在便宜的聊天視窗寫好，省 token；真正吃 token 的寫程式工作，才丟給 Agent。先把需求聊清楚，再交給 Agent 執行，大幅減少來回修改。

【P21 · Case02 · 成品 + QR】 ⌚ 40s (精簡)

這是成品——FuBao 成都之旅助手，已經上線。用了之後我發現，**它比我用過的所有旅遊 App 都好用——因為它完全按我的習慣做的。**

地圖整合、行程編輯、熊貓 UI 全都有。當你發現自己五小時就能做一個完全合用的 App，「買軟體訂閱」這件事，你就會重新思考。各位可以掃這個 QR，在手機上實際操作看看。

【P22 · Case02 · iframe 雙版】 ⌚ 10s 跳過

一樣是桌面加手機雙版展示，同一份行程即時同步，這頁我直接翻過。

【P23 · Case03 · 修電腦場景】 ⌚ 45s

第三個案例，是個很多人都遇過的痛：**「程式打不開。」**

看這個 error——MSVCR100.dll 缺失。以前我會怎麼處理？Google 搜尋、看五篇 2018 年的老部落格、還搞不清楚要裝 x86 還是 x64、然後下載一個來路不明的 DLL fixer、最後懷疑那個 fixer 是不是病毒.....平均花三十分鐘，還不一定修好。

【P24 · Case03 · 修電腦影片】 ⌚ 60s 播片

【起頭】這次我換個方式——直接讓 Codex 自己動手修。看這段影片，它自己診斷缺哪個 DLL、自己判斷位元數、**下載官方安裝包還先驗數位簽章才執行**，最後自己驗證修好。全程一分五十秒，在我的 PowerShell 裡跑。

(播放影片)

【收尾】注意它驗簽章那一步——這是連很多都會忽略的資安動作，它自己做了。

【P25 · Case03 · 4 動作精煉】 ⌚ 15s 一句帶過

拆開來看就四個動作：診斷、釐清、驗證、執行。一句話總結這個案例的意義：**以前 AI 是「告訴你怎麼修」，現在 AI 是「它幫你修好了」。**

【P26 · Ch6 章扉】 ⌚ 35s

最後一個章節。前面講的都是「工具」，但這一個月我最依賴的兩個東西——Hermes 跟 OpenClaw——**已經不只是工具，是助理。**

它們 24 小時在線、聽我指令、幫我做事。接下來兩個案例，你會看到它們有多誇張。

【P27 · OpenClaw 機場追星】 ⌚ 70s (故事)

這個故事有點好笑但很真實。我在大阪機場，想找出 aespa 搭哪一班——對，我是去追星的。

我沒有自己一班一班查，我請 OpenClaw 幫我找。看這兩張卡：aespa 搭韓亞航空 OZ114，從首爾飛大阪，18:30 落地；而我搭德威航空 TW313，從大邱飛大阪，18:25 落地——**同一個機場、同一個入境口，只差五分鐘。**

OpenClaw 做的事：即時查當下落地的所有航班、自動排除廉航、幫我鎖定 aespa 那班。中間這段影片，就是我在大阪機場現場，**真的遇到 aespa** 的畫面。

我不是查航班——我是請一個 Agent 幫我「找出 aespa」。它幫我把這件事做成了。

【P28 · Hermes Lemonade 演示影片】 ⌚ 60s 播片

【起頭】再看一個更直接的。這次是 Hermes，我躺在床上，對著手機講一句話：

「幫我用 Chrome 打開 YouTube，搜尋 aespa 最新的公開舞台影片，然後從 1 分 20 秒開始播放。」

看影片——它自己開 Chrome、進 YouTube、判斷「最新公開舞台」、跳轉到 1:20、播放。

(播放影片)

【收尾】全程我沒碰電腦、沒碰滑鼠，**只用嘴巴講了一句話。**

【P29 · 整合關係圖】 ⌚ 40s (精簡)

把這條神經串起來看：我用 **Hermes** 在手機上講話，這是指令層；透過 **MCP** 這個通訊協定，傳給在電腦上跑的 **Claude Code**，這是執行層；它在我的電腦上改檔、開瀏覽器，**事情就真的做完了。**

24 小時待命、我給指令、它做事。過去這是科幻片，今年，它在我家。

【P30 · 心法】 ⌚ 40s

講到這裡，我想留四句話給各位：

1. **不需要先學會，才能開始。**
2. **打開電腦，張開嘴。**
3. **剩下的，路上學。**

4. 踏出你的第一步。

我不是工程師，我就是這樣一步步走過來的。你也可以。

【P31 · 本週試試看】 ⌚ 45s

不要讓今天的分享只停在「聽聽就好」。我給各位三件本週就能做的事：

1. **下載 Typeless**——按住說話，鬆開字就出現，三分鐘、手機就能試。
2. **描述一個生活困擾**——說給 AI 聽，看它能不能幫你做出來，十分鐘、不用寫程式。
3. **發 Mail 或 Teams 私訊我**——告訴我一個你想解決的小問題，你不用自己動手，我來幫你評估。

重點不是要你變成工程師，而是讓你知道：**你離「擁有自己的工具」，只差開口一句話。**

【P32 · 致謝 Q&A】 ⌚ 30s

今天的分享就到這裡。這份簡報有線上版，掃這個 QR 就能看，裡面所有 demo 都能實際點。

言出法隨，幻化成真。希望今天的分享，能讓你想到一件自己也想試試看的事。

謝謝大家，有任何問題，現在或會後都歡迎，我的信箱在這。Q&A 時間。

📌 執行提醒（自己看，不用講）

- **三段影片 (P5 / P24 / P28) 是節奏關鍵**——讓影片自己講，你只起頭 + 收尾，別在播放時蓋話。
- **真正可砍的緩衝**：P11、P12、P14、P18、P22、P25 這六頁都是「一句帶過」，現場時間緊就用翻頁速度過，省 1.5 ~ 2 分鐘。
- **若超時**：優先再砍 P7 (想寫做) 和 P13 (五層工作流)，P6 已經把核心概念講過，這兩頁是延伸。
- **若還有餘裕**：P21 成品、P27 追星故事可以多講 10 ~ 20 秒，這兩個最有記憶點。